

TC-APP-SL: system_log — 시스템 로그 수집·압축·업로드 검증

목적 (Objective)

`system_log` 애플리케이션의 로그 수집, xz 압축, toupload 이관, Azure Blob 업로드, 30일 보존, Factory Reset, 리부트 전 로그 저장 등 전 기능을 검증한다. IPC(MQTT 브릿지)를 통한 on-demand export와 24시간 주기 rotation을 포함한다.

공통 전제 조건 (Common Preconditions)

- DUT 전원 ON, 네트워크 연결, SSH 또는 시리얼 콘솔(COM7, 115200 8N1) 접속 가능
- DUT에서 `system_log` 프로세스 실행 중 (`pgrep -f system_log`)
- MQTT 브로커 동작 중 (`localhost:1883`)
- `mosquitto_pub` / `mosquitto_sub` 설치됨
- `/edge/log/` 파티션 쓰기 가능

TC01 — 파일명 규칙

목적

생성된 `.log.xz` 파일명이 `systemlog_{14자리}_{14자리}.log.xz` 형식이며, 시작 시각 ≤ 저장 시각 조건을 만족하는지 확인한다.

사전 조건

- 공통 전제 조건 충족
- toupload 디렉토리(`/edge/log/toupload/system/`) 쓰기 가능
- `task_rotate_sync()` 실행 시 파일 생성 가능 상태 (디스크 여유 5MB 이상)

절차

1. SETUP: `get_log_data` 요청 전송 → `task_rotate_sync()` 실행 → toupload에 `.log.xz` 생성
2. `ls -t /edge/log/toupload/system/systemlog_*.log.xz | head -1` 로 최신 파일 획득
3. 파일명을 정규식 `systemlog_[0-9]{14}_[0-9]{14}\.log\.xz` 로 검증
4. 파일명에서 start(앞 14자리), end(뒤 14자리) 추출 후 `start <= end` 비교

기대 결과

항목	기준
파일명 형식	<code>systemlog_YYYYMMDDHHMMSS_YYYYMMDDHHMMSS.log.xz</code>
시각 순서	<code>start ≤ end</code>

PASS/FAIL Criteria

기준 ID	설명	타입	기준 값	셀 검증
TC01-1	파일명 정규식 일치	boolean	true	<code>grep -qE "systemlog_[0-9]{14}_[0-9]{14}\\.log\\.xz"</code>
TC01-2	시작 ≤ 저장 시 각	boolean	true	<code>["\$start_t" -le "\$end_t"]</code>

TC02 — 24시간 타이머

목적

`system_log` 타이머 루프가 24시간 경과 시 `task_rotate_sync()`를 실행하여 toupload에 `.log.xz` 파일을 생성하는지 확인한다.

사전 조건

- 공통 전제 조건 충족
- 시스템 시간 변경 권한 (root)
- NTP 자동 동기화 정지/복원 권한 (`timedatectl set-ntp false/yes`, `systemctl start/stop chronyd systemd-timesyncd`)
- `journalctl -u docker-loader`에서 `[system_log_timer_loop] loop started` 라인 확인 가능 — 즉 `system_log` 어플리케이션의 timer thread 가 부팅 직후 정상 시작되어 24h 주기 check 루프가 돌고 있는 상태
- **TC02** 는 다른 TC 들의 **SETUP**(`get_log_data`)보다 먼저 실행되어야 함 — **SETUP**의 `task_rotate_sync()` 호출이 timer thread 의 `last_run_time` 갱신에 영향을 줄 수 있어 +25h shift 후에도 `elapsed < 24h` 가 되어 발화 누락 가능
- 환경변수: 없음 (TC 진입 시 자동으로 NTP off)

절차

1. `journalctl -u docker-loader --no-pager | grep '[system_log_timer_loop] loop started'` 로 timer thread 시작 로그 확인
2. `FILES_BEFORE` = 현재 toupload `.log.xz` 파일 수 및 최신 파일명 기록
3. 시스템 시간을 현재 시간과 동기화 (NTP `set-ntp yes` → 잠시 대기 → `set-ntp false` 로 변경 가능 상태)
4. 현재 epoch `t0` 저장 후 시스템 시간을 `t0 + 25*3600` 로 변경 (`date -s @<epoch>`)
5. 타이머 발화 대기 (70초) — `system_log_timer_loop` 의 1초 `sleep_for` + `elapsed >= 24h` check 후 `task_rotate_sync()` 호출 완료 대기
6. toupload 에 신규 `.log.xz` 파일 생성 확인 및 파일명의 `endtime` 이 변경한 시간(+25h) 근처인지 확인
7. 시스템 시간을 현재 시간으로 복원 (NTP `set-ntp yes`)

기대 결과

항목	기준
toupload 파일	신규 <code>.log.xz</code> 생성됨

항목

기준

파일명 endtime

변경한 시스템 시간(+25h) 근처

PASS/FAIL Criteria

기준 ID	설명	타입	기준 값	셀 검증
TC02-1	toupload 신규 .xz 생성	boolean	true	["\$files_after" -gt "\$files_before"]
TC02-2	파일명 endtime이 변경 시간 근처	manual	—	파일명 확인 후 수동 판정

TC03 — On-demand export

목적

`get_log_data` IPC 요청 수신 시 `task_rotate_sync()`를 비동기 스레드로 실행하고, 완료 후 MQTT 응답 (`error_code=0`)과 신규 `.log.xz` 파일 생성을 확인한다.

사전 조건

- 공통 전제 조건 충족
- toupload 디렉토리 쓰기 가능
- system_log MQTT 토픽 구독 가능 (`emsp/system_log/+/req/get_log_data`)

절차

- `FILES_BEFORE` = 현재 `/edge/log/toupload/system/systemlog_*.log.xz` 수
- `mosquitto_sub` 구독 시작 → `mosquitto_pub` 로 `get_log_data` 송신 → 응답 대기 (30초)
- 응답 수신 후 10초 추가 대기 (파일 생성은 detached thread에서 비동기 진행)
- `FILES_AFTER` 재카운트 → 파일 수 증가 확인

구현 주의: `task_rotate_sync()`는 detached thread에서 실행되어 MQTT 응답 반환 후 비동기로 파일 생성이 완료된다. 응답 수신만으로 파일 존재를 보장하지 않으므로 응답 후 추가 대기가 필요하다.

기대 결과

항목	기준
응답 수신	MQTT 응답 수신 (30초 이내)
신규 파일	<code>FILES_AFTER > FILES_BEFORE</code> (응답 후 10초 이내)

PASS/FAIL Criteria

기준 ID	설명	타입	기준 값	셀 검증
-------	----	----	------	------

기준 ID	설명	타입	기준 값	셀 검증
TC03-1	xz 파일 신규 생성 됨	boolean	true	["\$FILES_AFTER" -gt "\$FILES_BEFORE"]

TC04 — On-demand timeout (실제 journal 데이터 시나리오)

목적

journald 가 실제로 기록한 데이터로 /edge/log/system/journal/<machine-id>/ 가 **100MB / 300MB** 사이즈일 때 `get_log_data` 요청이 IPC 타임아웃 내에 응답을 반환하고 10초 이내에 `.log.xz` 파일까지 생성되는지를 확인한다.

단순히 `dd` 로 zero-fill 한 더미 `.journal` 은 journald 가 corrupted 로 즉시 무시하므로 시나리오 의도 (대용량 journal 처리 시 timeout 검증)를 측정할 수 없다. 따라서 `systemd-cat` 으로 journald 에 실제 데이터를 주입해 valid journal 파일을 만든다.

사전 조건

- 공통 전제 조건 충족
- `journalctl --rotate` 및 `--vacuum-files` 권한 (root)
- `systemd-cat` 사용 가능 (journald 가용)
- `journald.conf`: `SystemMaxFileSize` 기본 64M, `SystemMaxFiles` 기본 20 — 300MB 까지 ~5개 파일 필요, 한도 안에 들어감
- 디바이스 emmc 가용 공간 500MB 이상 (300MB journal + 압축 작업 임시공간)
- IPC 타임아웃: `SYSTEM_LOG_REQUEST_CMD_TIMEOUT=5초`, `SYSTEM_LOG_PUBLISH_TIMEOUT=7초`

절차

2개 사이즈(100MB / 300MB)에 대해 다음을 반복:

1. `journalctl --rotate && journalctl --vacuum-files=1` 로 journal 초기화
2. `before_files` = 현재 toupload `.log.xz` 파일 수
3. `head -c <raw>MB /dev/urandom | base64 -w 4096 | systemd-cat -t TC04_DUMMY` 로 실제데이터 주입
 - 사이즈별 raw urandom 양: 70MB / 210MB (≈1.4x 팽창 후 journal 목표 사이즈에 도달)
4. `sync; sleep 3; journalctl --rotate; sleep 2` 로 디스크에 flush
5. `journalctl --disk-usage` 로 실제 journal 사이즈 확인
6. `get_log_data` 요청 송신, 응답 epoch 차이로 응답시간 측정
7. 10초 추가 대기 후 `after_files` 재카운트
8. [`"$after_files" -gt "$before_files"`] 로 PASS/FAIL 판정

마지막 사이즈 시험 후: `journalctl --rotate && journalctl --vacuum-files=1` 로 디스크 복원.

알려진 제약: journal 사이즈가 100MB 이상이면 `journalctl -o cat | xz` 파이프라인이 `SYSTEM_LOG_REQUEST_CMD_TIMEOUT=5초` 안에 끝나지 않고 SIGKILL 로 실패할 수 있다. 이 케이스에서는 `error_code=UNKNOWN` 으로 응답이 반환되고 `.xz` 가 생성되지 않는다. FAIL 시 `system_log` 로그의 `task_rotate_sync / request_command_sync` 흔적과 `exit_code=-1` 응답을 확인한다.

기대 결과

항목	기준
각 사이즈 응답	MQTT 응답이 30초 안에 반환 (대용량에서는 <code>error_code=UNKNOWN</code> 도 응답으로 인정)
파일 생성	가능하면 10초 이내 <code>.log.xz</code> 신규 생성 (대용량에서 FAIL 시 알려진 제약으로 노트)

PASS/FAIL Criteria

기준 ID	설명	타입	기준 값	셀 검증
TC04-1	journal 100MB 상태에서 10초 이내 .xz 파일 생성	boolean	true	["\$after" -gt "\$before"]
TC04-2	journal 300MB 상태에서 10초 이내 .xz 파일 생성	boolean	true	["\$after" -gt "\$before"]

TC05 — xz 압축

목적

rotation 완료 후 생성된 파일이 유효한 `.xz`이며, 원본 `.log` 파일이 삭제되었는지 확인한다.

사전 조건

- 공통 전제 조건 충족
- TC03 또는 TC04 직후 (toupload에 신규 `.log.xz` 1개 이상 존재)
- 디바이스에 `xz --test` 명령 사용 가능

절차

- `LATEST_XZ = ls -t /edge/log/toupload/system/systemlog_*.log.xz | head -1`
- `[ -f "$LATEST_XZ" ]` 확인
- `xz --test "$LATEST_XZ"` 실행 → exit code 0 확인
- `LOG_FILE="{LATEST_XZ%.xz}" → [ ! -f "$LOG_FILE" ]` 확인

기대 결과

항목	기준
.xz 파일 존재	toupload에 파일 있음
무결성	<code>xz --test</code> exit 0
원본 .log	삭제됨

PASS/FAIL Criteria

기준 ID	설명	타입	기준 값	셀 검증
-------	----	----	------	------

기준 ID	설명	타입	기준값	셀 검증
TC05-1	.xz 파일 존재	boolean	true	[-f "\$LATEST_XZ"]
TC05-2	xz 무결성	exit code	0	xz --test "\$LATEST_XZ"
TC05-3	원본 .log 삭제	boolean	true	[! -f "\${LATEST_XZ%.xz}"]

TC06 — Journal rotation

목적

`task_rotate_sync()` 완료 후 `journalctl --rotate && journalctl --vacuum-files=1` 실행 결과로 저널 디스크 사용량이 감소하는지 확인한다.

사전 조건

- 공통 전제 조건 충족
- `journalctl --disk-usage` 사용 가능
- TC03 SETUP 직후 (rotation 트리거된 상태)
- 환경변수: `SYSTEM_LOG_CMD_ROTATE_VACUUM="journalctl --rotate && journalctl --vacuum-files=1"`

제약: FW 업데이트 시 machine-id가 바뀌어 이전 부팅의 저널 파일이 별도 서브디렉토리에 잔존한다. vacuum은 현재 machine-id만 처리하므로 전체 파일 수는 줄지 않을 수 있다. 저널 사용량(용량) 감소로 확인한다.

절차

- SETUP 전 `journalctl --disk-usage` 로 용량 기록 (`JOURNAL_SIZE_BEFORE`)
- TC03 SETUP (`get_log_data`) 실행 → rotation 완료
- `journalctl --disk-usage` 재측정 (`JOURNAL_SIZE_AFTER`)
- 수동으로 사용량 감소 또는 이미 최소 상태임을 확인

기대 결과

항목	기준
저널 사용량	감소하거나 이미 최소 상태

PASS/FAIL Criteria

기준 ID	설명	타입	기준값	셀 검증
TC06-1	rotate && vacuum 실행 확인	manual	사용량 감소 또는 최소 상태	<code>journalctl --disk-usage</code> 수동 확인

TC07 — 30일 보존 정책

목적

`delete_log()` 가 `mtime > 30`일 파일을 삭제하고, 30일 미만 파일은 보존하는지 확인한다.

사전 조건

- 공통 전제 조건 충족
- `touch -d "31 days ago" / "29 days ago"` 명령 사용 가능 (mtime 조작)
- 환경변수: `LOG_RETAIN_DAY=30` (system_log 빌드 상수)

절차

1. `SETUP(get_log_data)` 완료 후 TC07 진입 시 더미 파일 생성:

```
touch -d "31 days ago"
/edge/log/toupload/system/systemlog_20250101000000_20250101010000.log.
xz
touch -d "29 days ago"
/edge/log/toupload/system/systemlog_20250501000000_20250501010000.log.
xz
```

- 2. `get_log_data` 재요청 → `task_rotate_sync()` → `delete_log()` 호출됨 (10초 대기)
- 3. 31일 더미 파일 존재 여부 확인 (`[ ! -f ... ]`)
- 4. 29일 더미 파일 존재 여부 확인 (`[ -f ... ]`)

주의: 더미 파일을 `SETUP` 이전에 생성하면 `task_rotate_sync()`가 더미 삭제(-1)와 신규 파일 생성(+1)을 동시에 수행해 TC03의 파일 수 순증가가 0이 되므로, TC07 전용으로 별도 `get_log_data` 호출을 통해 삭제를 트리거한다.

기대 결과

항목	기준
31일 경과 파일	삭제됨
29일 경과 파일	유지됨

PASS/FAIL Criteria

기준 ID	설명	타입	기준값	셀 검증
TC07-1	31일 파일 삭제	boolean	true	<code>[! -f dummy_31]</code>
TC07-2	29일 파일 유지	boolean	true	<code>[-f dummy_29]</code>

TC08 — Azure Connector 업로드 확인

목적

`task_rotate_sync()` 완료 후 `toupload` 디렉토리에 `.log.xz` 및 `.meta` 파일이 존재하여 `azure_connector`가 업로드할 준비가 됐는지 확인한다.

사전 조건

- 공통 전제 조건 충족
- TC03 또는 TC07 직후 (`toupload`에 `.xz` 1개 이상 존재)
- `azure_connector / blob_upload_director` 실행 여부와 무관 (업로드 자체는 TC 범위 밖)

범위: `system_log`의 책임(`toupload` 이관)만 검증한다. 실제 Azure Blob 전송 성공 여부는 이 TC의 범위 밖이다.

절차

1. TC03 SETUP 완료 후 (`get_log_data` 응답 수신)
2. `/edge/log/toupload/system/systemlog_*.log.xz` 존재 확인
3. `/edge/log/toupload/system/systemlog_*.log.xz.meta` 존재 확인

기대 결과

항목	기준
<code>.log.xz</code>	<code>toupload</code> 에 존재
<code>.log.xz.meta</code>	<code>toupload</code> 에 존재

PASS/FAIL Criteria

기준 ID	설명	타입	기준 값	셀 검증
TC08-1	<code>toupload</code> 에 <code>.log.xz</code> 존재	boolean	true	<code>ls /edge/log/toupload/system/systemlog_*.log.xz</code>
TC08-2	<code>toupload</code> 에 <code>.meta</code> 존재	boolean	true	<code>ls /edge/log/toupload/system/systemlog_*.log.xz.meta</code>

TC09 — Factory Reset

목적

`request_factory_reset` IPC 요청 수신 시 `/edge/log/toupload/system/` 내 모든 파일이 삭제되는지 확인한다.

사전 조건

- 공통 전제 조건 충족
- `toupload` 디렉토리에 1개 이상의 파일 존재 (사전 더미 또는 직전 TC 결과 사용 가능)

- system_log MQTT 토픽 발행 권한 (emsp/system_log/+/req/request_factory_reset)

절차

1. 더미 파일 생성: touch /edge/log/toupload/system/systemlog_dummy.log.xz
2. mosquitto_pub → request_factory_reset 요청, 30초 대기
3. 응답 수신 확인
4. 더미 파일 + 디렉토리 내 모든 파일 소멸 확인

기대 결과

항목	기준
응답	error_code = 0 수신
toupload 파일	전체 삭제

PASS/FAIL Criteria

기준 ID	설명	타입	기준 값	셀 검증
TC09-1	factory_reset 응답 수신	boolean	true	[-n "\$resp"]
TC09-2	toupload 파일 전체 삭제	boolean	true	[-z "\$(ls \${TOUPLoad_DIR}/*. * 2>/dev/null)"]

TC10 — 리부트 전 로그 저장

목적

shutdown_application_for_system_reboot IPC 요청 시 shutdown 로그가 staging에 저장되고, 실제 리부트 후 boot log(부팅 시 무조건 캡처 + vacuum)와 합쳐져 toupload에 이관되는지 확인한다.

사전 조건

- 공통 전제 조건 충족
- DUT 실제 리부트 가능 환경 (테스트 종료 후 90~120초의 부팅 시간 허용)
- 시리얼 콘솔(COM7) 접속 권장 — SSH는 reboot 시 끊김
- staging(/edge/log/system/)과 toupload(/edge/log/toupload/system/) 쓰기 가능
- /edge/log/system/.tc10_before 임시 파일 작성 가능 (TC10-PRE의 toupload 개수 저장용)

절차

Phase 1 — 리부트 전 (--tc10-pre):

1. BEFORE_TOUPLoad = toupload .log.xz 파일 수 기록
2. mosquitto_pub → shutdown_application_for_system_reboot 요청, 60초 대기
3. 응답 수신 확인
4. staging .log.xz 생성 확인

5. **reboot** 실행 (SSH 연결 종료, 시리얼은 유지)
- Phase 2 — 리부트 후 (--tc10-post, 재접속 후 수동 실행):**

1. **AFTER_TOUPLoad** = toupload .log.xz 파일 수 확인 (boot log 무조건 캡처 + merge 결과)

2. 파일 수 증가 확인 (**AFTER** > **BEFORE_TOUPLoad**)

3. staging 비워짐 확인

기대 결과

항목	기준
응답 수신 (pre)	MQTT 응답 수신
staging .xz (pre)	신규 생성됨
toupload .xz (post)	파일 수 증가

PASS/FAIL Criteria

기준 ID	단계	설명	타입	기준 값	셀 검증
TC10-1	pre	응답 수신	boolean	true	[-n "\$resp"]
TC10-2	pre	staging .xz 생성	boolean	true	ls /edge/log/system/systemlog_*.log.xz
TC10-3	post	toupload 파일 수 증가 (boot 캡처 + merge)	boolean	true	["\$AFTER_TOUPLoad" -gt "\$BEFORE_TOUPLoad"]

TC11 — nmon 업로드 happy path

목적

task_upload_nmon()이 /edge/log/system/nmon/old/*.nmon 를 /edge/log/toupload/system/nmon/ 으로 이동하고 .meta(post_action_success=delete, post_action_failure=move, move_dir_failure=/edge/log/system/nmon/archive, from=system_log, upload_path=/ems-system/nmon/YYYY/MM/) 를 생성하는지, 그리고 BlobUploadDirector 5분 스캔이 toupload 항목을 정상 처리하는지 검증한다.

사전 조건

- 공통 전제 조건 충족
- /edge/log/system/nmon/old/ 쓰기 가능 (없으면 mkdir)
- /edge/log/toupload/system/nmon/ 디렉토리 쓰기 가능 (task_upload_nmon 이 lazy 생성)
- 디바이스가 Azure Blob 정상 통신 가능 — TC11-5 검증에 필요 (실패 시 후처리 자동 archive 이동으로 알려진 동작)

- system_log MQTT 토픽 발행 권한 (emsp/system_log/+/req/get_log_data) — TC03 트리거 재 사용

절차

1. /edge/log/system/nmon/old/ 비우고 더미 .nmon 3개 생성 (dummy_tc11_a.nmon, dummy_tc11_b.nmon, dummy_tc11_c.nmon) — 각 파일에 헤더 라인 1줄 기록
2. baseline 카운트 — INPUT_COUNT=3, TOUPLoad_BEFORE = /edge/log/toupload/system/nmon/*.nmon 수
3. send_and_wait "get_log_data" "{}" 30 으로 SERVICE_GET_LOG_DATA 트리거 (TC03 패턴 재사용)
4. 응답 후 5초 대기 — task_upload_nmon() 의 fs::rename + create_upload_task 완료 보장
5. 즉시 단계 검증:
 - /edge/log/system/nmon/old/ 의 .nmon 수가 0인지
 - /edge/log/toupload/system/nmon/ 에 .nmon + .nmon.meta 페어 3쌍 존재하는지
 - 임의 .nmon.meta 1개를 grep 하여 4개 필드 + upload_path 매치

기대 결과

항목	기준
/edge/log/system/nmon/old/*.nmon	0개 (입력 전체 이동됨)
/edge/log/toupload/system/nmon/	입력 개수만큼 .nmon + .nmon.meta 페어
.meta upload_path	/ems-system/nmon/YYYY/MM/ (현재 연/월)
.meta 후처리 필드	post_action_success=delete, post_action_failure=move, move_dir_failure=/edge/log/system/nmon/archive, from=system_log

PASS/FAIL Criteria

기준 ID	설명	타입	기준 값	셀 검증
TC11-1	/edge/log/system/nmon/old/ 의 .nmon 0개	boolean	true	["\$old_after" -eq 0]
TC11-2	toupload 의 .nmon 갯수가 trigger 전·후로 증가 (단순 증가만 확인)	boolean	true	["\$xfer_after" -gt "\$xfer_before"]
TC11-3	.meta 의 upload_path=/ems-system/nmon/YYYY/MM/ 매치	boolean	true	grep -qE "^upload_path=/ems-system/nmon/\${yyyy}/\${mm}/" "\$any_meta"
TC11-4	.meta 의 4개 후처리 필드 모두 매치	boolean	true	4 grep 모두 0

기준 ID	설명	타입	기준 값	셸 검증
TC11-5	toupload 의 .nmon.meta 갯수가 trigger 전·후로 증가 (단순 증가만 확인)	boolean	true	["\$meta_after" -gt "\$meta_before"]

TC12 — nmon retention 30일

목적

nmon.sh 의 find -mtime +30 -exec rm -f 가 nmon/old, nmon/archive, toupload/system/nmon 3개 디렉토리 모두에서 정상 동작하는지 확인.

사전 조건

- 공통 전제 조건 충족
- 위 3개 디렉토리 쓰기 가능 (없으면 mkdir)
- touch -d "40 days ago" 명령 사용 가능 (mtime 조작)
- systemctl restart nmon.service 권한 (root)

절차

1. 3개 디렉토리에 더미 파일 생성:
 - 40일 더미: tc12_old40.nmon, tc12_old40.nmon.meta 등 디렉토리당 1쌍
 - 현재 시각 더미: tc12_now.nmon, tc12_now.nmon.meta 등 디렉토리당 1쌍
 - 40일 더미는 touch -d "40 days ago" 로 mtime 조작
2. systemctl restart nmon.service 실행 (nmon.sh 재실행 → retention 블록 발화)
3. 3초 대기 (nmon.sh 의 find 실행 완료 보장)
4. 3개 디렉토리에서 더미 존재/부재 확인

기대 결과

항목	기준
40일 더미	3개 디렉토리 모두 부재
현재 시각 더미	3개 디렉토리 모두 존재

PASS/FAIL Criteria

기준 ID	설명	타입	기준 값	셸 검증
TC12-1	3 디렉토리에서 40일 mtime .nmon 모두 삭제	boolean	true	3 디렉토리 모두 [! -f "\$old40"]
TC12-2	3 디렉토리에서 현재 시각 .nmon 보존	boolean	true	3 디렉토리 모두 [-f "\$now_file"]

기준 ID	설명	타입	기준 값	셸 검증
TC12-3	3 디렉토리에서 40일 mtime .nmon.meta 모두 삭제	boolean	true	3 디렉토리 모두 [! -f "\$old40_meta"]
TC12-4	3 디렉토리에서 현재 시각 .nmon.meta 보존	boolean	true	3 디렉토리 모두 [-f "\$now_meta"]

TC13 — nmon 부재 환경 호환 (no-op)

목적

/edge/log/system/nmon/old/ 가 비어있거나 디렉토리 자체가 미존재일 때, task_upload_nmon() 이 에러 없이 (응답 error_code=0) 동작하는지 확인.

사전 조건

- 공통 전제 조건 충족
- /edge/log/system/nmon/old/ 비울 권한 (root)
- system_log MQTT 토픽 발행 권한 (TC03 트리거 재사용)

절차

1. /edge/log/system/nmon/old/ 내부 *.nmon / *.nmon.meta 전부 제거 (디렉토리 자체는 남김 — 환경 친화 케이스)
2. send_and_wait "get_log_data" "{}" 30 으로 트리거 → 응답 수신 확인
3. 응답 페이로드에 error_code 추출 후 0 확인 (없으면 응답 자체 수신만으로 PASS — TC03 와 동일 정책)

기대 결과

항목	기준
MQTT 응답	30초 이내 수신
에러	없음 (error_code=0 또는 응답 수신)

PASS/FAIL Criteria

기준 ID	설명	타입	기준 값	셸 검증
TC13-1	get_log_data 응답 수신 (nmon old 비어있어도 에러 없음)	boolean	true	[-n "\$resp"]
TC13-2	응답 페이로드에 error_code 가 있으면 0 또는 "NONE" (둘 다 정상 의미, task_rotate_sync 정상)	boolean	true (있을 시)	echo "\$resp" \ grep -qE '"error_code"[[[:space:]]]*: [[[:space:]]]*(0\ "NONE")' (또는 필드 부재 시 skip)

기준 ID	설명	타입	기준 값	셀 검증
TC13-3	최근 1분 journald 에 [task_upload_nmon] ERROR/Failed 로그 부재 (silent failure 가드)	boolean	true	journalctl -u docker-loader --since "1 minute ago" \ grep -F '[task_upload_nmon]' \ grep -E 'ERROR\ Failed' 결과 빈 문자열

환경 변수 (Environment Variables)

변수	기본값	설명
MQTT_HOST	localhost	MQTT 브로커 주소
SOURCE	tc_runner	MQTT 발신 source ID
TARGET	system_log	MQTT 수신 대상 앱 ID
TOUPLOAD_DIR	/edge/log/toupload/system	toupload 경로
NMON_OLD_DIR	/edge/log/system/nmon/old	nmon 회전 완료 파일 위치 (TC11/TC12/TC13)
NMON_TOUPLOAD_DIR	/edge/log/toupload/system/nmon	nmon toupload 경로 (TC11/TC12)
NMON_ARCHIVE_DIR	/edge/log/system/nmon/archive	nmon 업로드 실패 시 이동 디렉토리 (TC12)

디렉토리 구조 참고

```
/edge/log/
├── system/                                ← STAGING_DIR (systemlog.sh 기동 시 mkdir)
│   ├── systemlog_A_B.log.xz             ← shutdown/boot 캡처 파일 (부팅 후 toupload 이관)
│   │   └── archive/                     ← Azure 업로드 실패 시 lazy 생성
│   └── toupload/
│       └── system/                       ← TOUPLOAD_DIR (task_rotate_sync 또는 merge 후 이관)
│           ├── systemlog_X_Y.log.xz
│           └── systemlog_X_Y.log.xz.meta
```

자동화 등급 (Automation Grade)

□ B

TC	등급	비고
----	----	----

TC	등급	비고
TC01, TC03~TC09	A (자동)	무인 실행 가능
TC02	A (자동)	시스템 시간 ±25h 자동 변경 + 복원
TC04	A (자동)	systemd-cat으로 100/300MB 실 journal 데이터 주입 + vacuum cleanup
TC06	B (반자동)	저널 사용량 수동 확인
TC10	B (반자동)	실제 리부트 포함 — pre/post 분리 실행, 재접속 후 post 수동 실행
TC11	B (반자동)	nmon 업로드 happy path — TC11-5 는 5분+ 대기 (BlobUploadDirector 스캔)
TC12	A (자동)	nmon retention 30일 — <code>systemctl restart nmon.service</code> 후 3 초 확인
TC13	A (자동)	nmon old 비어있는 환경 호환 — <code>get_log_data</code> 응답 수신만 확인

관련 문서

- `tc_system_log_result.md` — 본 TC 실행 결과 보고서
- `tc_system_log_evidence_full.log` — 결과의 근거가 되는 통합 로그